Lecture 2: Fundamentals & Tabular ML

41-501
Applied AI

etienne@cmkl.ac.th

Class Organization (Preliminary)

- 29 August: Introduction
- 05 September: Fundamentals & Tabular ML
- 12 September: Computer Vision
- 19 September: Recommender Systems & RL
- (26 September: OFF)
- 03 October: Evaluation & Production
- 10 October: Demo Day?

Assignment

- 19 September: Midterm
- 03 October: deadline submission
- 10 October: Final Assignment/Demo Day
- 12-16 October: 10 minute in person

- Or: lecture on 10 October, demo in person 12-16 October

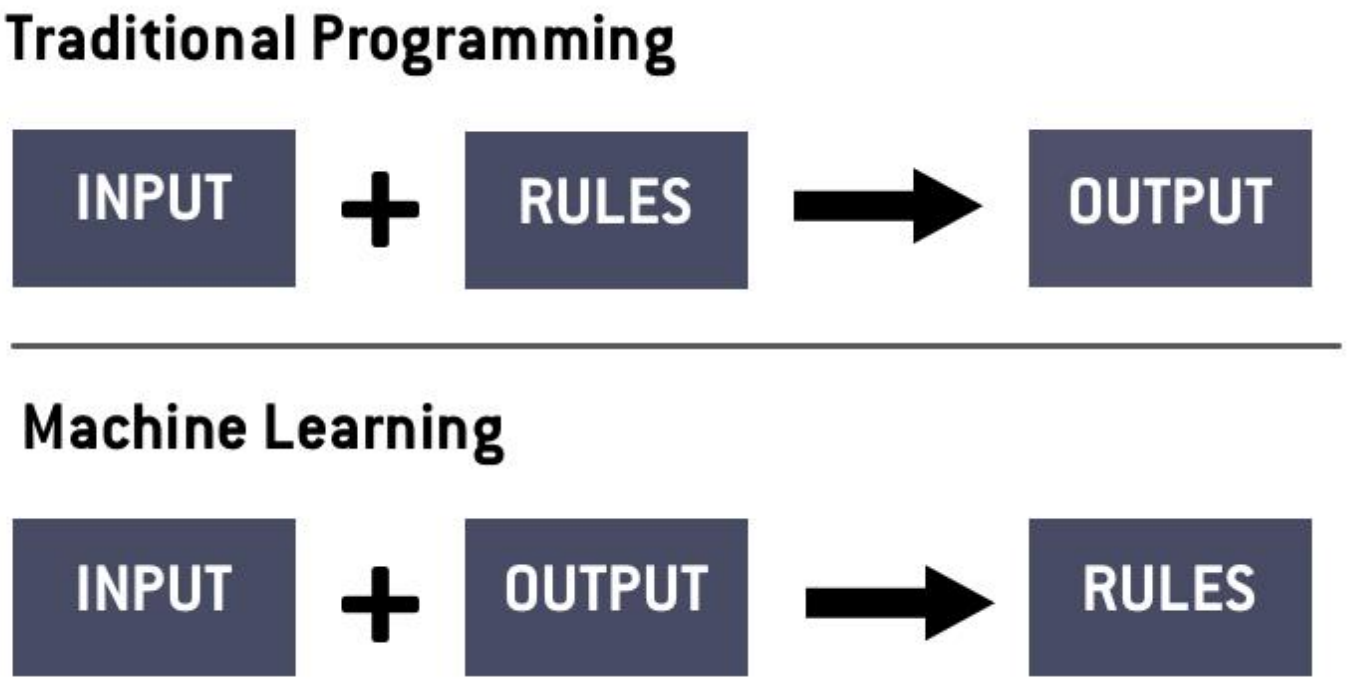
Assignment

- Midterm Assignment: Project Proposal
  - You want money/time from: investor, boss, prof, etc
  - Evaluation of the TECHNICAL part
  - No AI slop (-> stand out)
- Final Assignment: Product demo
- In person: show understanding

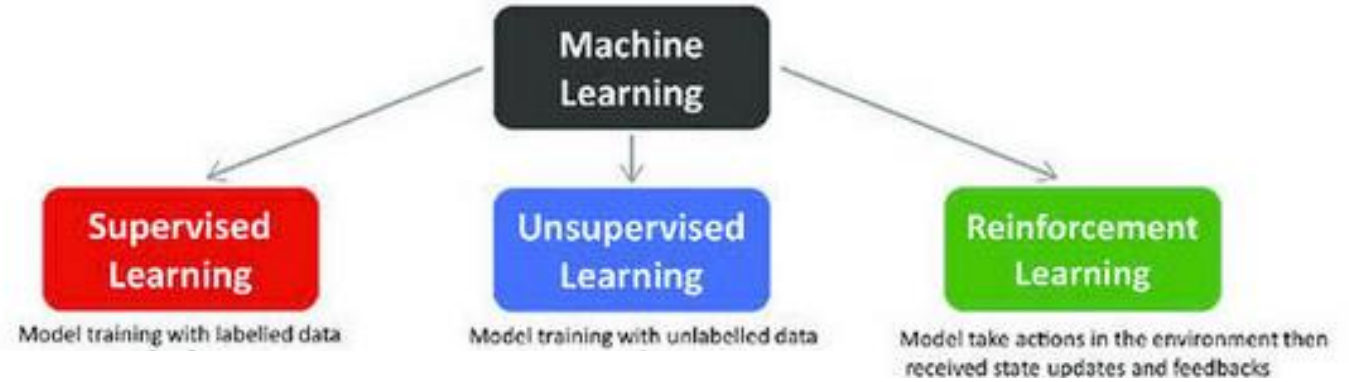
Overview

- ML Fundamentals
- Data Processing
- Tabular ML
- Lab

Traditional Programming vs ML

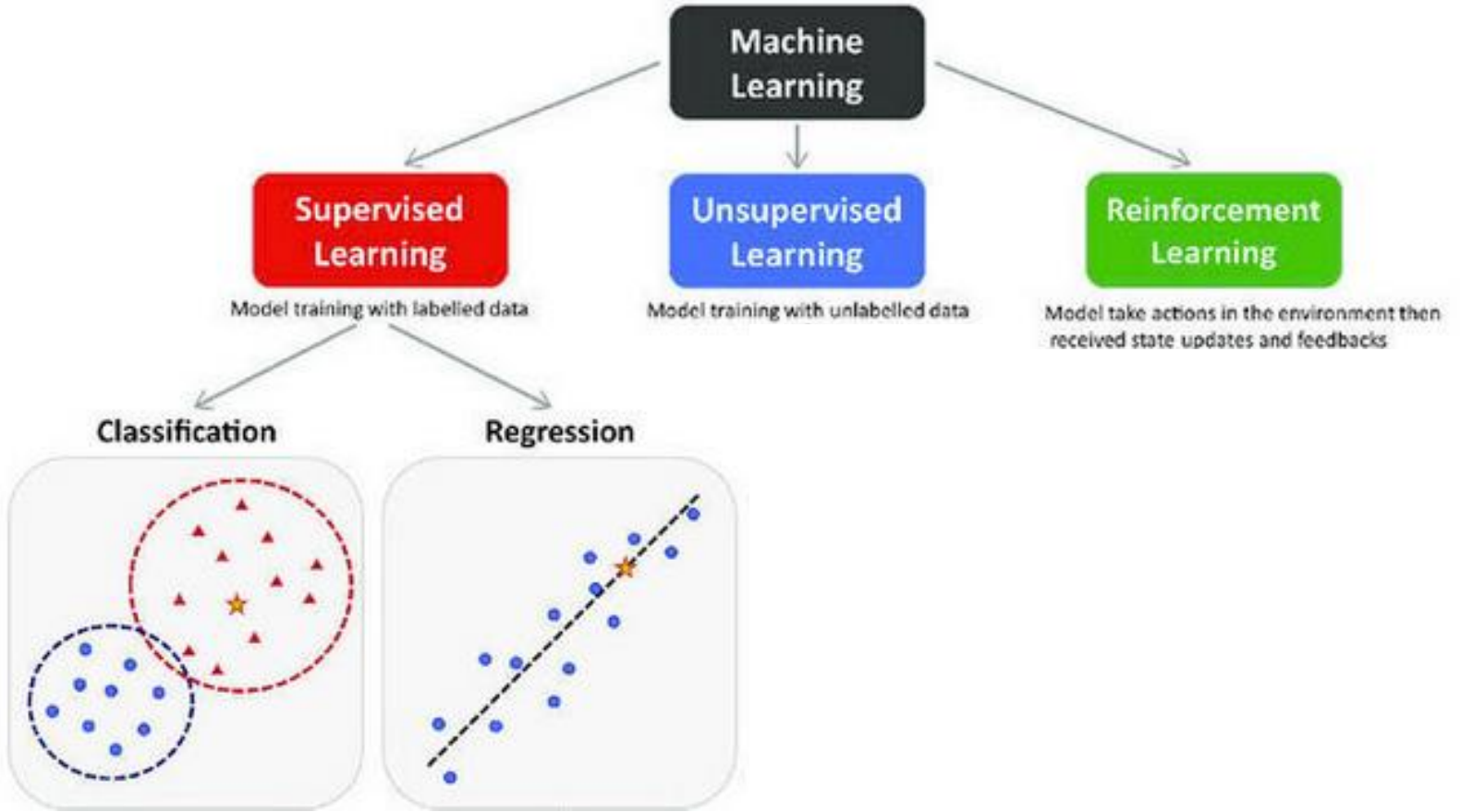


Three Types of Learning



- Supervised learning: you have labelled examples
- Unsupervised learning: you find structure without labels
- Reinforcement learning: an agent learns by trial and reward

Supervised Learning

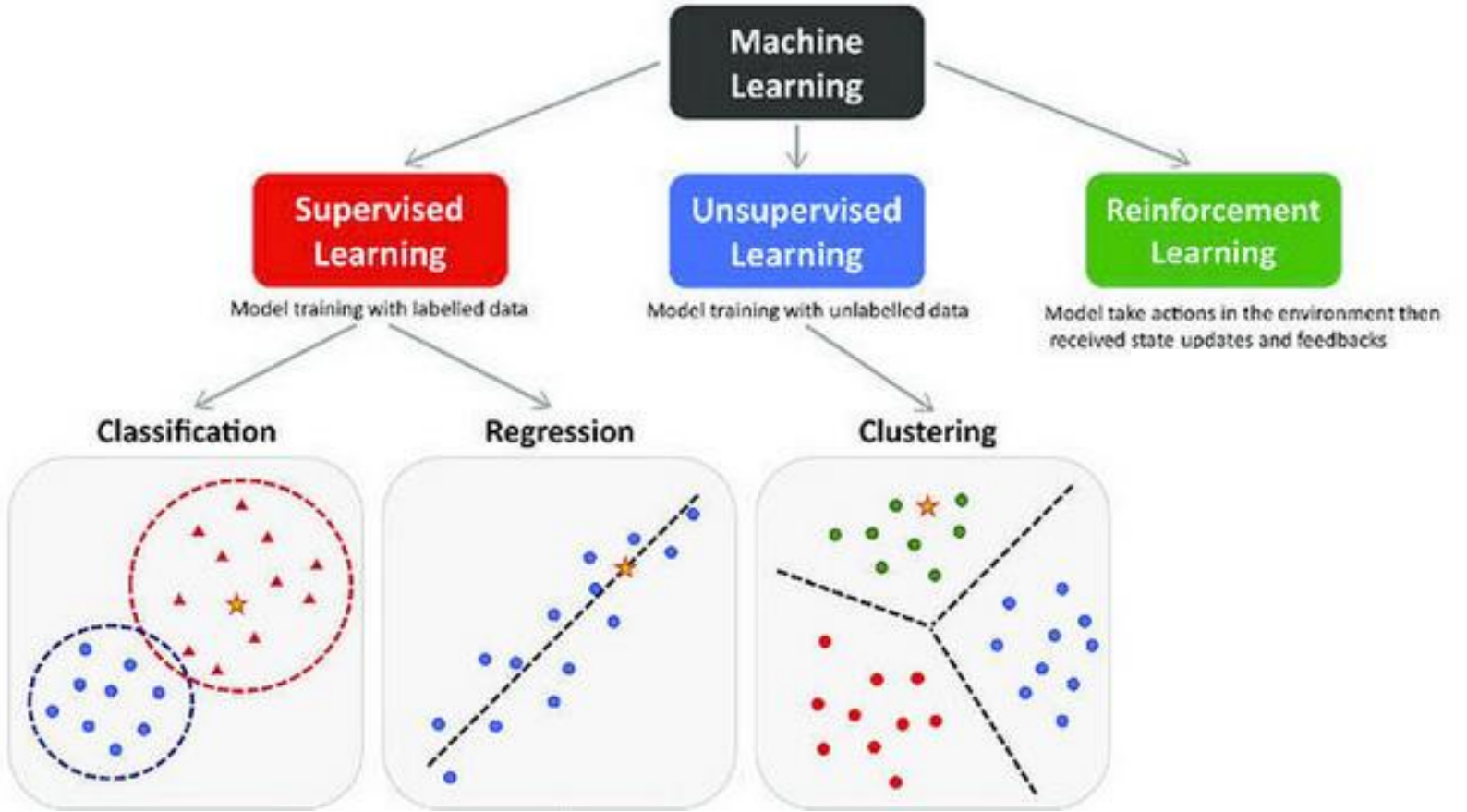


Given input X and known output Y, learn a mapping
$f(X) \rightarrow Y$

Two flavors:

- Classification: predict a category (spam or not, tumor type)
- Regression: predict a number (house price, temperature tomorrow)

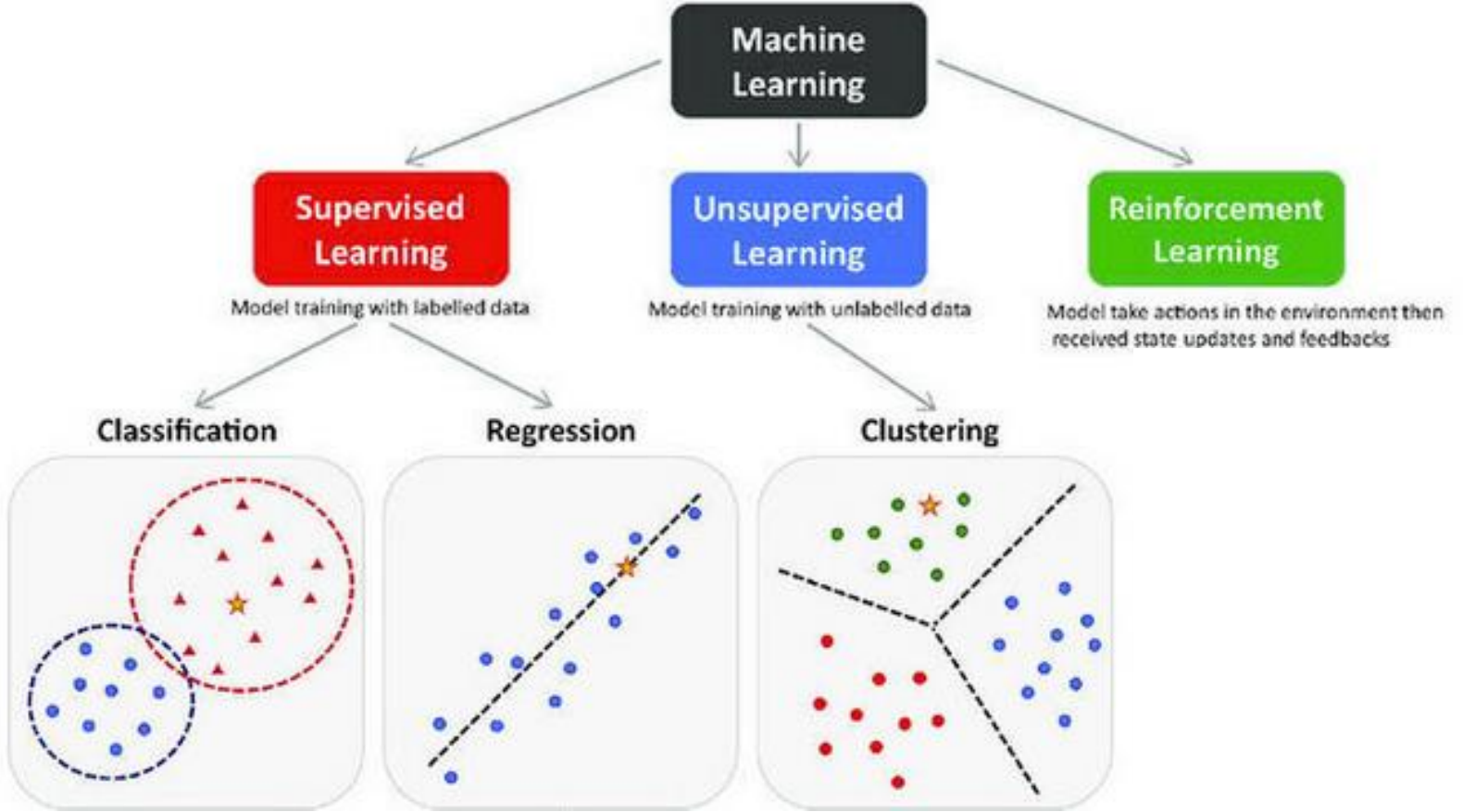
Unsupervised Learning



No labels: discover hidden structure

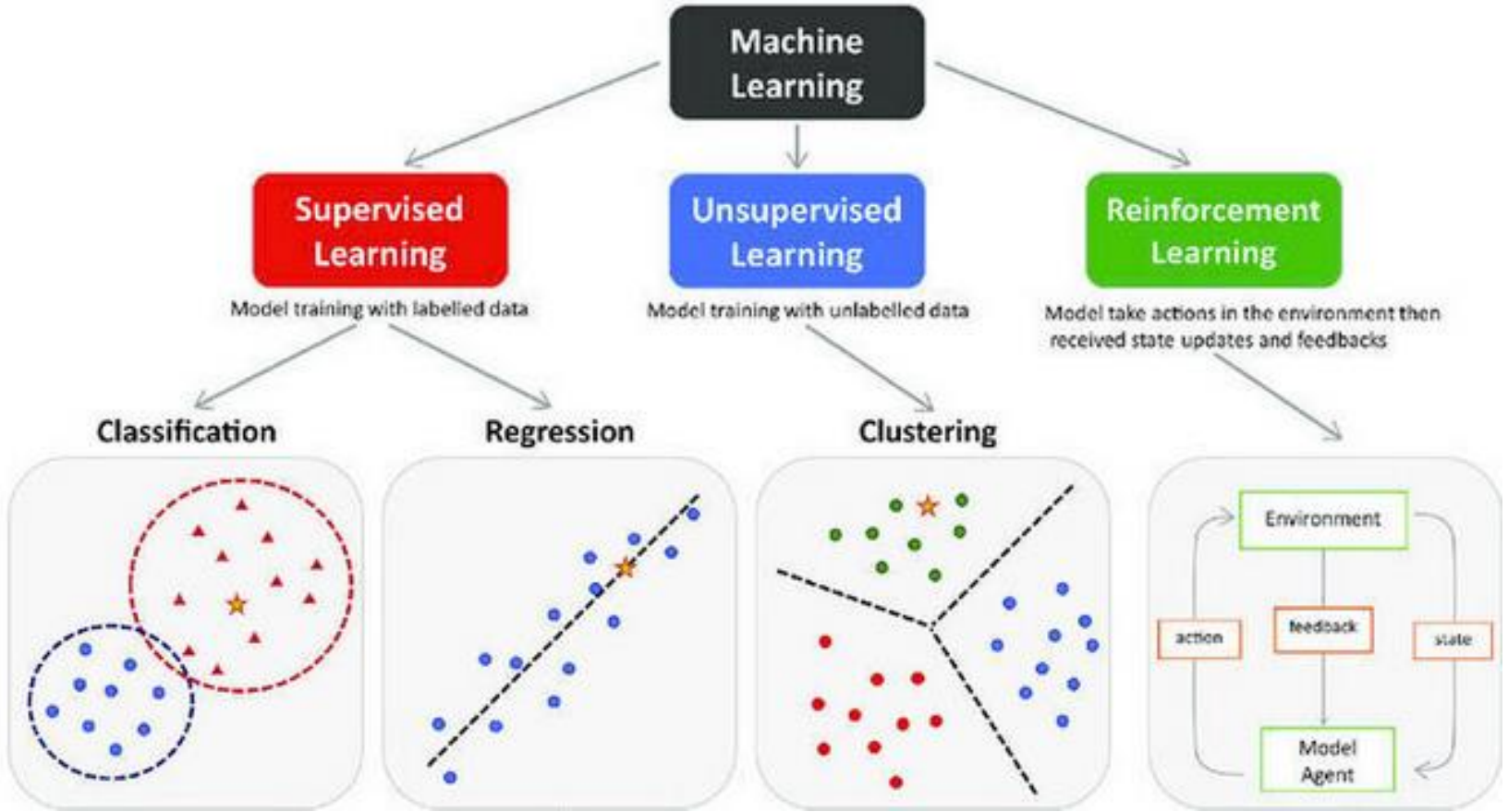
- Clustering: group similar items (customer segments, document topics)
- Dimensionality reduction: compress features while keeping signal (PCA, t-SNE)
- Anomaly detection: find things that don't belong

(Semi-Supervised & Self-Supervised)



- Semi-supervised: small set of labels + large set of unlabeled data
- Self-supervised: create labels from the data itself (e.g., predict the next word, mask part of an image)
- Why? Labelling is expensive, data is cheap

Reinforcement Learning



An agent takes actions in an environment and receives rewards or penalties

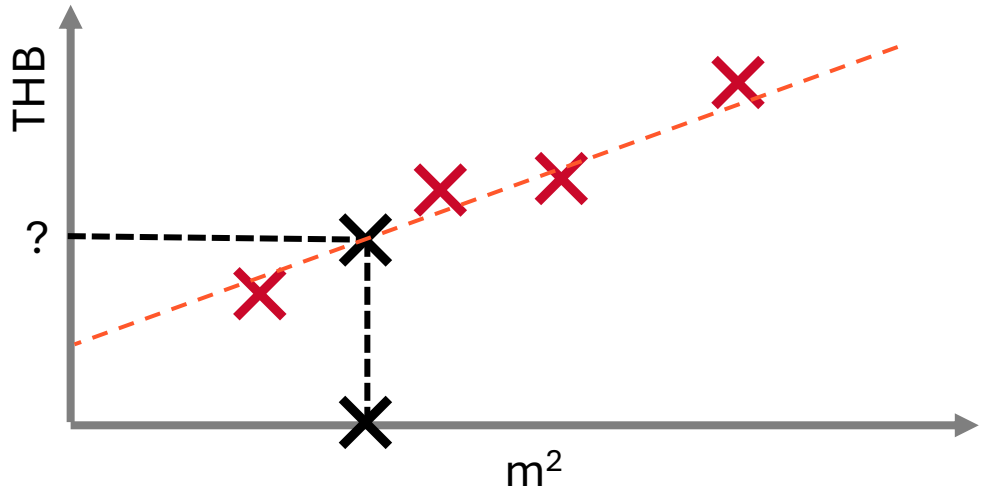
- No labeled data: learns by trial and error
- Goal: maximize cumulative reward over time
- Examples: game-playing AI, robotics control, ad placement optimization

How Does a Model Actually Learn?

1. Define a model with parameters
2. Define a loss function: "how wrong are we?”
3. Optimize: adjust parameters to reduce the loss
4. Repeat until good enough

Linear Regression: Housing Prices

Size [m^2]	Price [M THB]
79	8.2
55	5.3
50	5.1
32	2.9



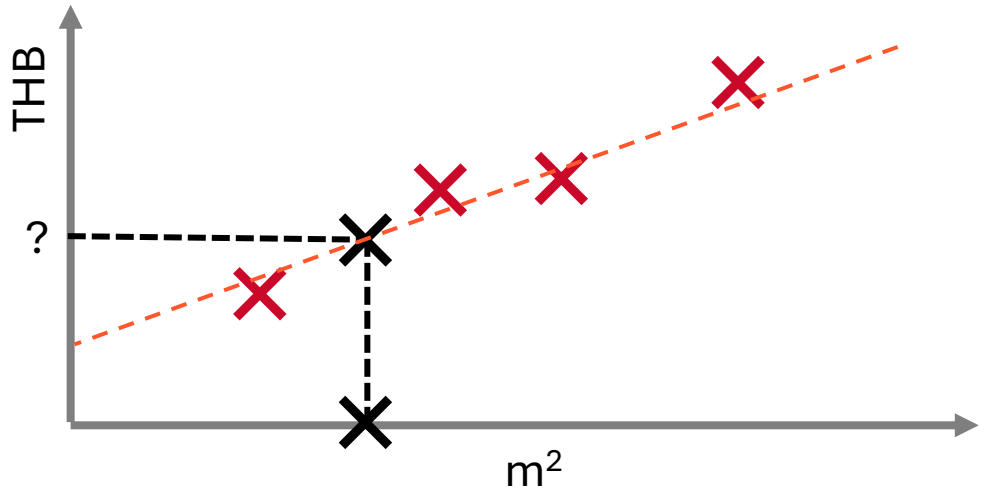
How do we predict housing prices?

Using a linear function!

$$f(x) = mx + b$$

Linear Regression: Housing Prices

x_1 (Size [m^2])	y (Price [M THB])
79	8.2
55	5.3
50	5.1
32	2.9



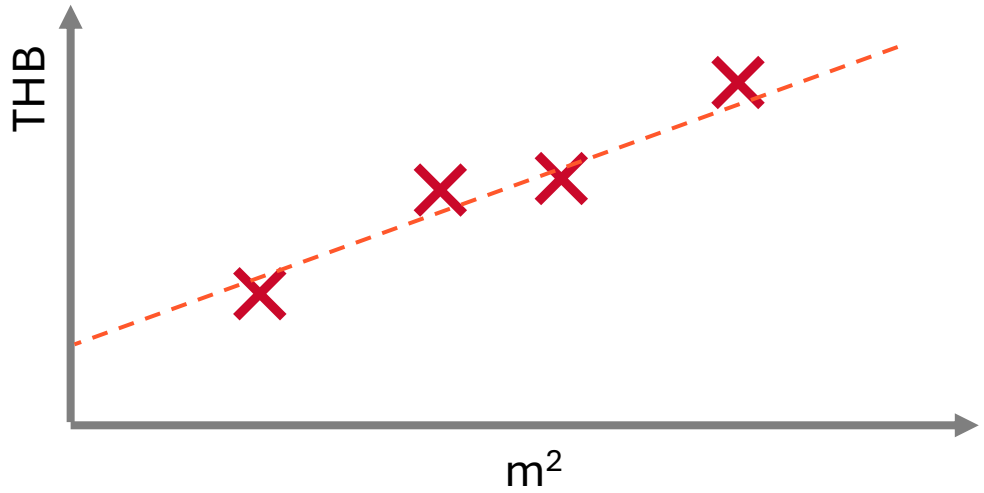
How do we predict housing prices?

Using a linear function!

$$\hat{y}(x) = w_1 x_1 + b$$

Linear Regression: Housing Prices

x_1 (Size [m^2])	x_2 (# Rooms)	y (Price [MTHB])
79	2	8.2
55	2	5.3
50	1	5.1
32	1	2.9



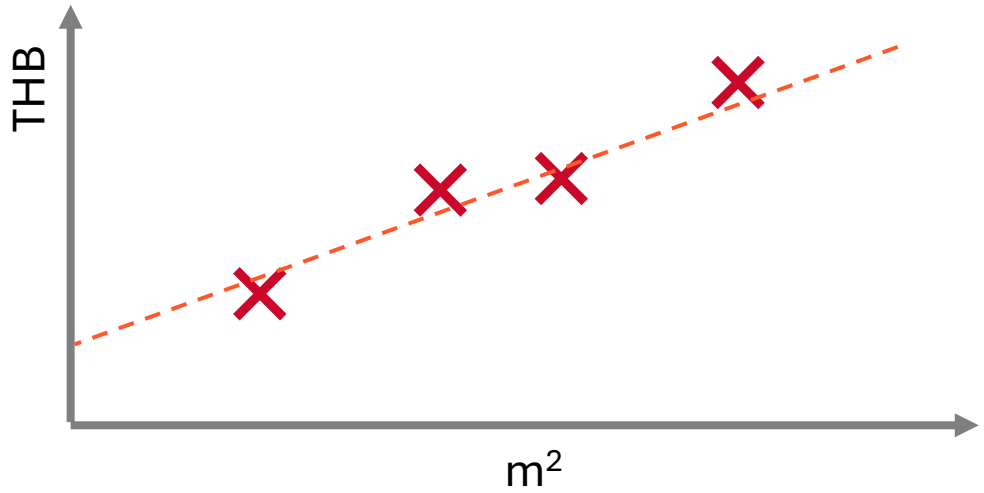
How do we predict housing prices?

Using a linear function!

$$\hat{y}(x) = w_1 x_1 + w_2 x_2 + b$$

Linear Regression: Housing Prices

x_1 (Size [m^2])	x_2 (# Rooms)	y (Price [MTHB])
79	2	8.2
55	2	5.3
50	1	5.1
32	1	2.9



How do we predict housing prices?

Using a linear function!

$$\hat{y}(x) = w_1 x_1 + w_2 x_2 + \cdots + w_n x_n + b$$

Loss Function

$$J(w) = \frac{1}{2}\sum_{i=1}^{m}(\hat{y}(x) - y)^2$$

- Minimize difference between predicted output and target variable
- Square to make it a positive value
- Sum over training examples
- (1/2 to make the math simpler later on)
- $J(w)$ is the Cost function (here: Squared Error)

Different Losses for different tasks:

- Regression: *Mean Squared Error*, Mean Absolute Error
- Classification: Cross-entropy loss, Hinge loss

Gradient Descent

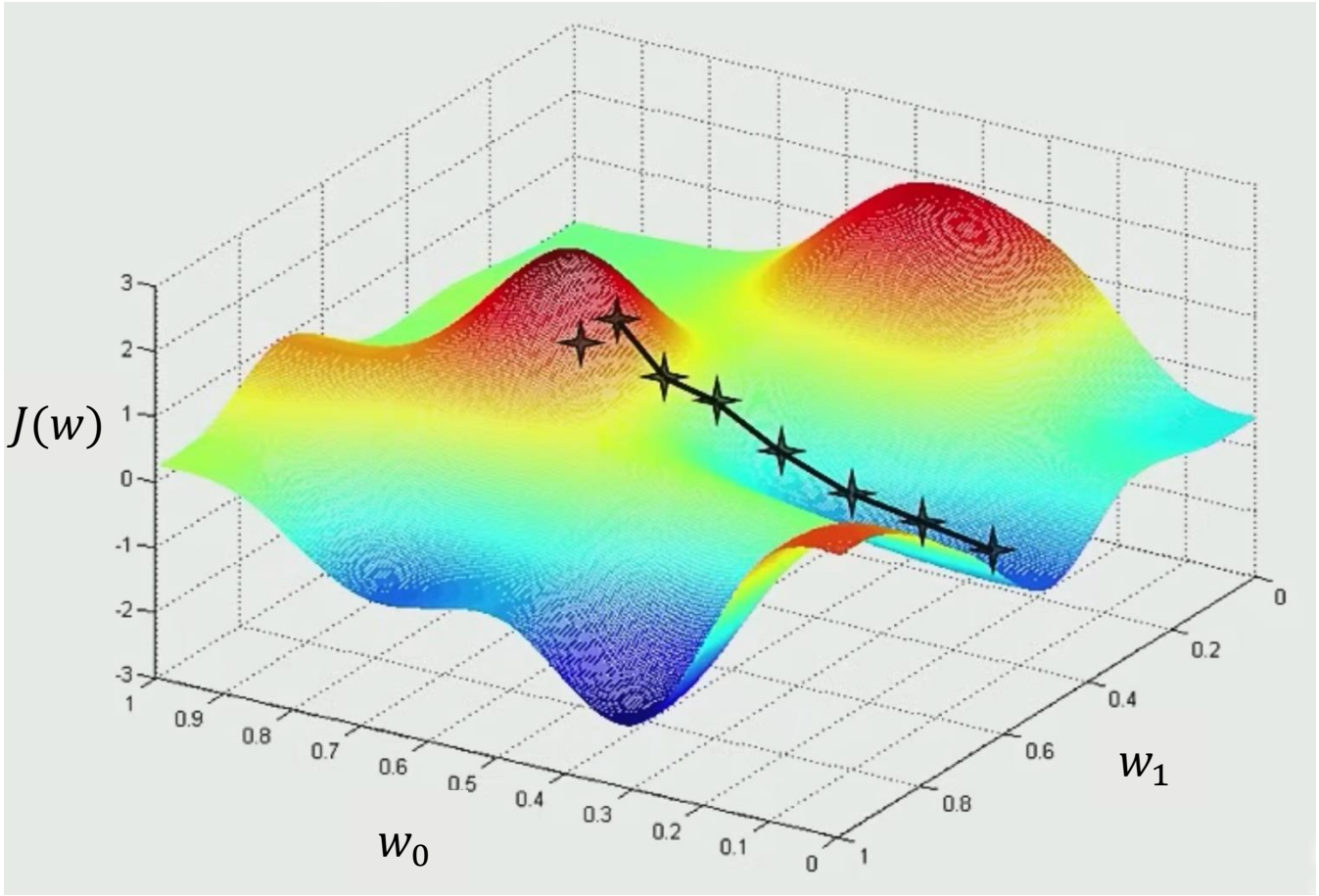
Goal:

- Choose $\vec{w}$ such that $\hat{y}(x) \approx y$ for training examples

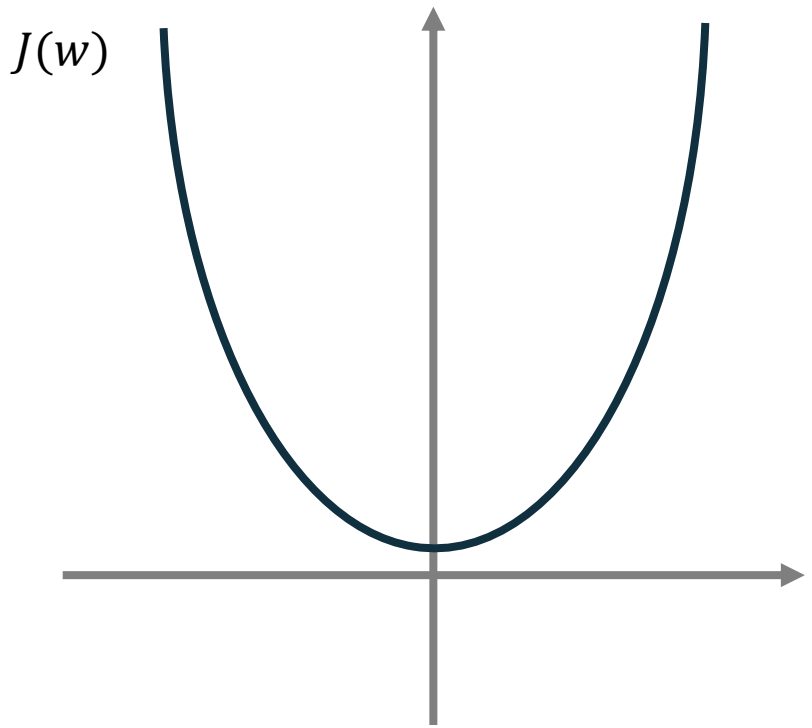
Method:

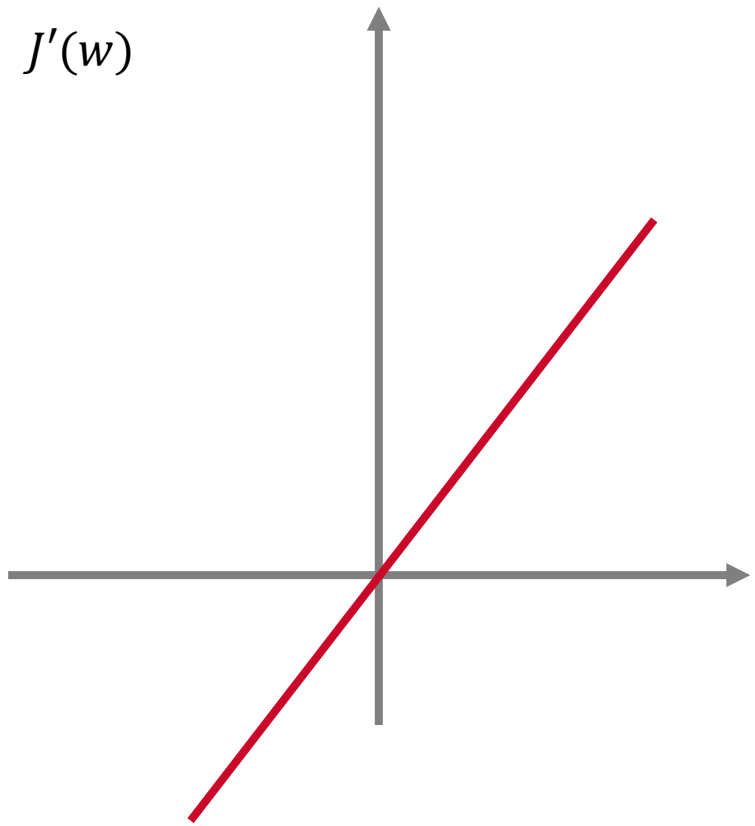
- Start with random value for $\vec{w}$
- Take a step towards lower gradient
- Keep changing $\vec{w}$ to minimize $J(w)$ until optima

Gradient Descent



Gradient Descent





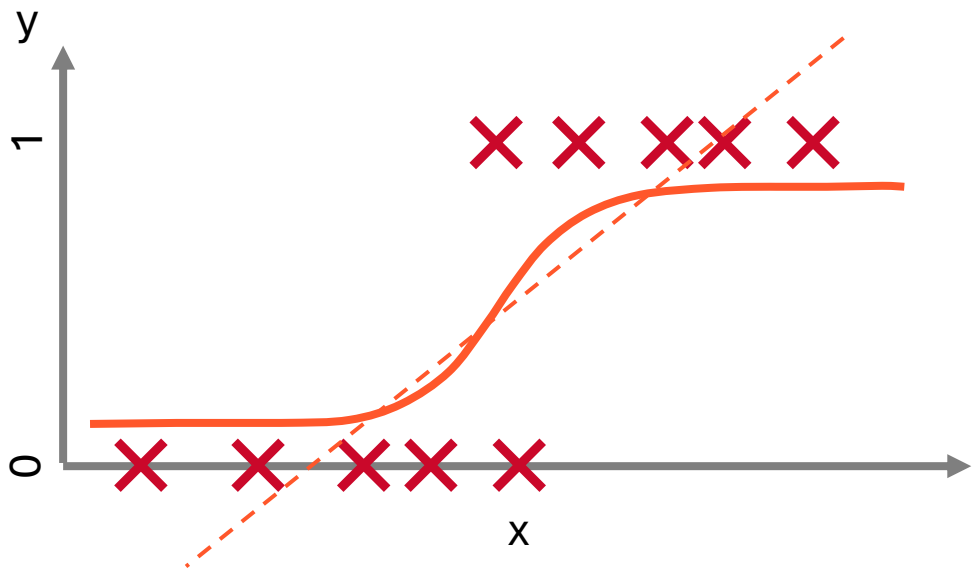
Gradient Descent

Update weights:

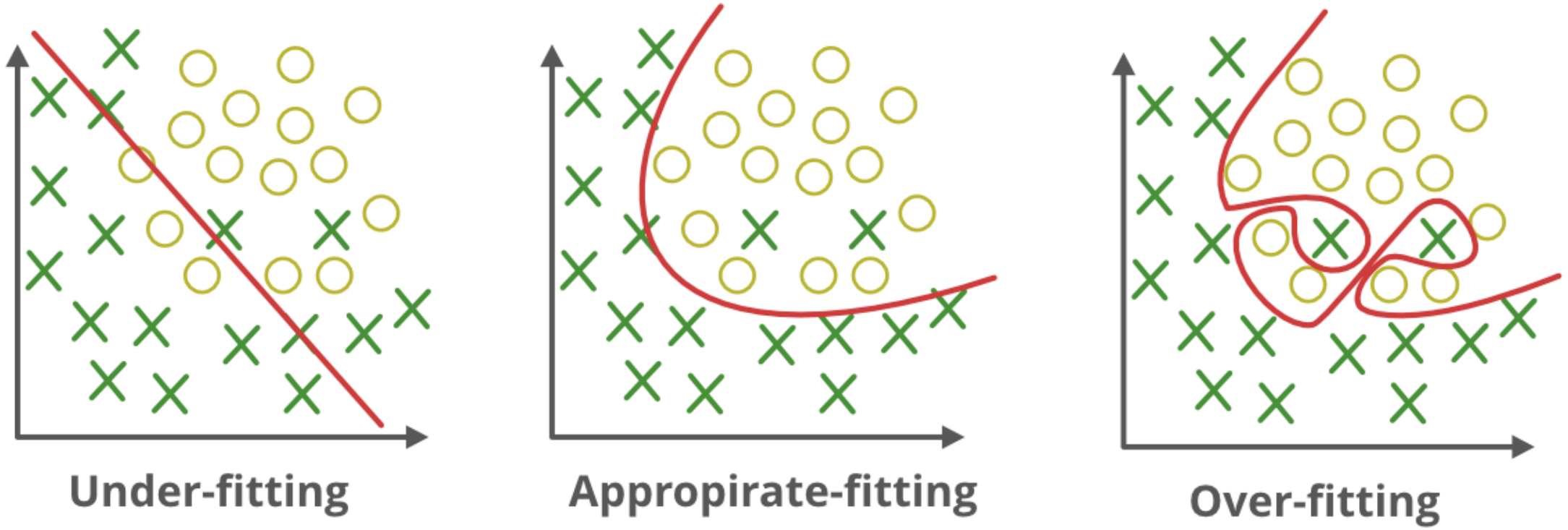
$$w_j := w_j - \alpha \frac{\partial}{\partial \mathrm{w}} J(w)$$

(α = learning rate, e.g. 0.01)

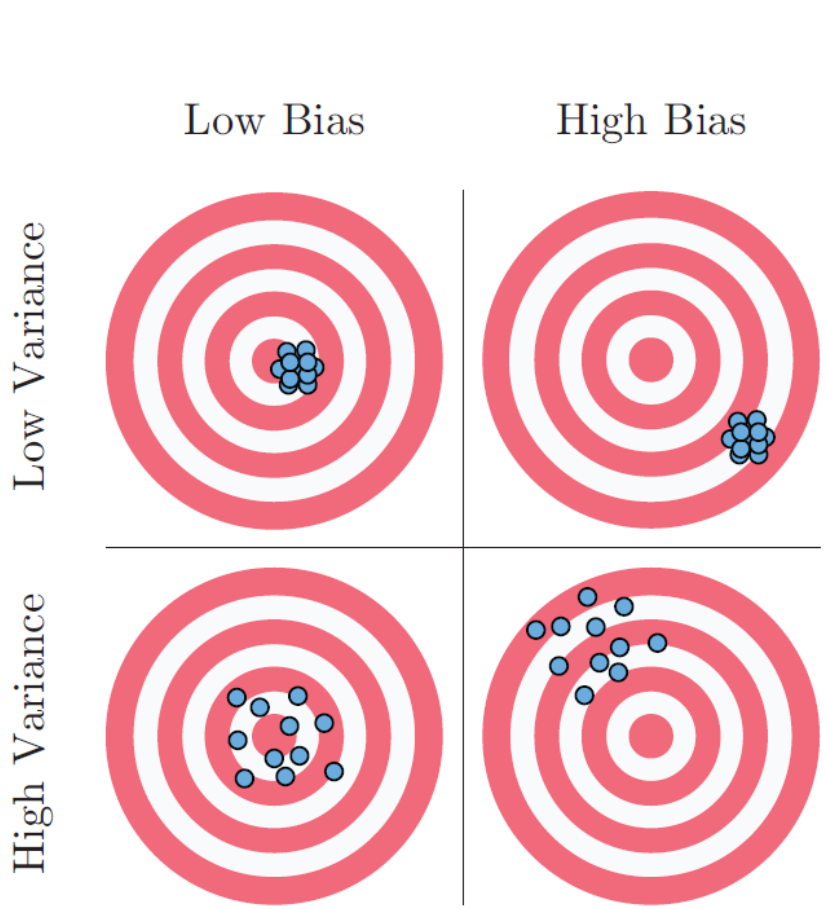
Classification

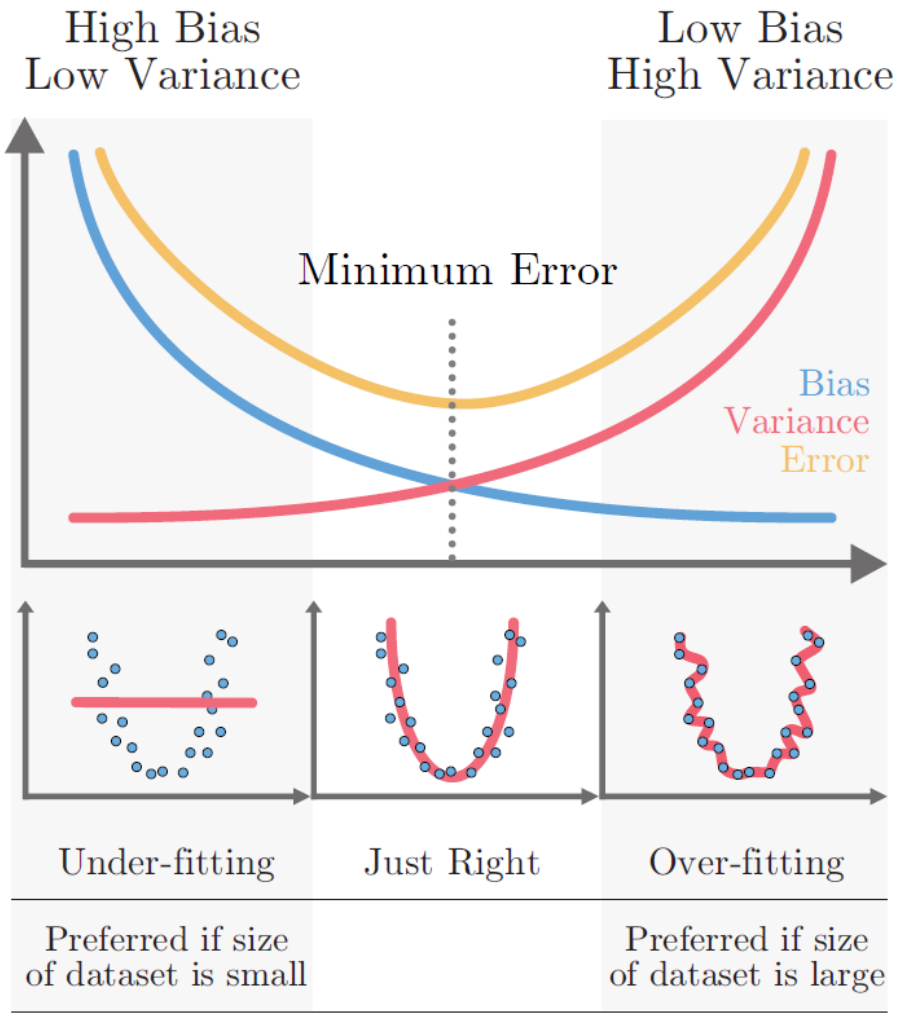


Underfitting vs Overfitting



The Bias-Variance Tradeoff

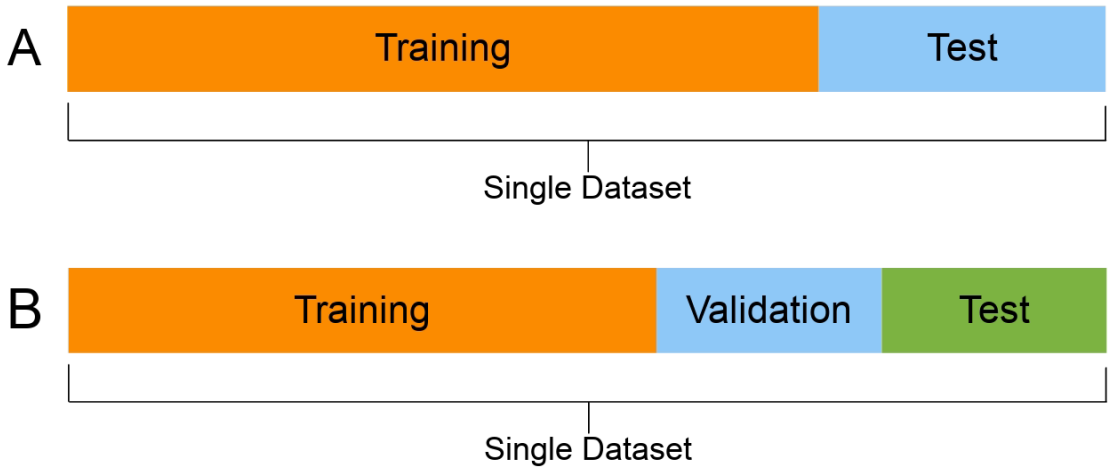




Train / Validation / Test Split

Model should generalize, not memorize!

- Training: The model learns from this
- Validation: Tune hyperparameters, monitor training
- Test: Final, honest evaluation (Not being used until the end)



Typical splits: 70/15/15 or 80/10/10 (dataset size matters)

Overview

- ML Fundamentals
- Data Processing
- Tabular ML
- Lab

Data Is the Hard Part

- Industry rule of thumb: 80% of ML work is data preparation
- “Garbage in, garbage out”
- Good models can't fix bad data!

Anatomy of a Tabular Dataset

x_1 (Size [m^2])	x_2 (# Rooms)	y (Price [MTHB])
79	2	8.2
55	2	5.3
50	1	5.1
32	1	2.9

- Rows = samples / observations
- Columns = features / variables
- Target = what you're predicting
- Feature types:
  - numerical (continuous, discrete)
  - categorical (nominal, ordinal)
  - text
  - datetime

Exploratory Data Analysis (EDA)

Before modelling, understand your data

- Shape & size: how many rows, how many features?
- Distributions: histograms, box plots for each feature
- Correlations: which features relate to each other and the target?
- -> Spot problems early

Handling Missing Values

First: understand why data is missing (random? systematic?)

Options:

- Drop rows: if few are missing, but you lose information
- Drop columns: if a feature is mostly empty
- Impute: fill with mean, median, mode, or a model-based prediction
- Flag: add a binary "was_missing" indicator
- (No universal answer)

Encoding Categorical Variables

- Models need numbers, not strings
- Label encoding: map categories to integers (dog=0, cat=1, bird=2): implies ordering, use for ordinal data
- One-hot encoding: create binary columns per category: no false ordering, but can explode dimensionality
- Target encoding: replace category with the mean of the target: powerful but risk of leakage

Feature Scaling

- Some algorithms are sensitive to feature magnitude (SVM, KNN, gradient descent), others are not (tree-based models)
- Standardization: mean=0, std=1
- Min-max scaling: rescale to [0, 1]
- Fit scaler on training data only, then transform validation/test

Feature Engineering

- Creating new features from existing ones
- Examples:
  - Extract day-of-week from date
  - Compute ratios between columns
  - Bin continuous variables
- -> Domain knowledge!!!
- Good features can matter more than model choice

Class Imbalance

- Problem: 95% negative, 5% positive
- -> model predicts "negative" always, gets 95% accuracy

Techniques:

- Resample: oversample minority (SMOTE) or undersample majority
- Class weights: tell the model to penalize mistakes on the rare class more
- Use the right metric: precision, recall, F1, AUC instead of accuracy

Data Leakage

- When information from outside the training set sneaks into the model
- Examples:
  - Scaling before splitting
  - Using future data to predict the past
  - Target leaking through a correlated feature
- Extremely common, extremely dangerous: your model looks great in the lab and fails in production
- Rule: every transformation must be fit only on training data

Overview

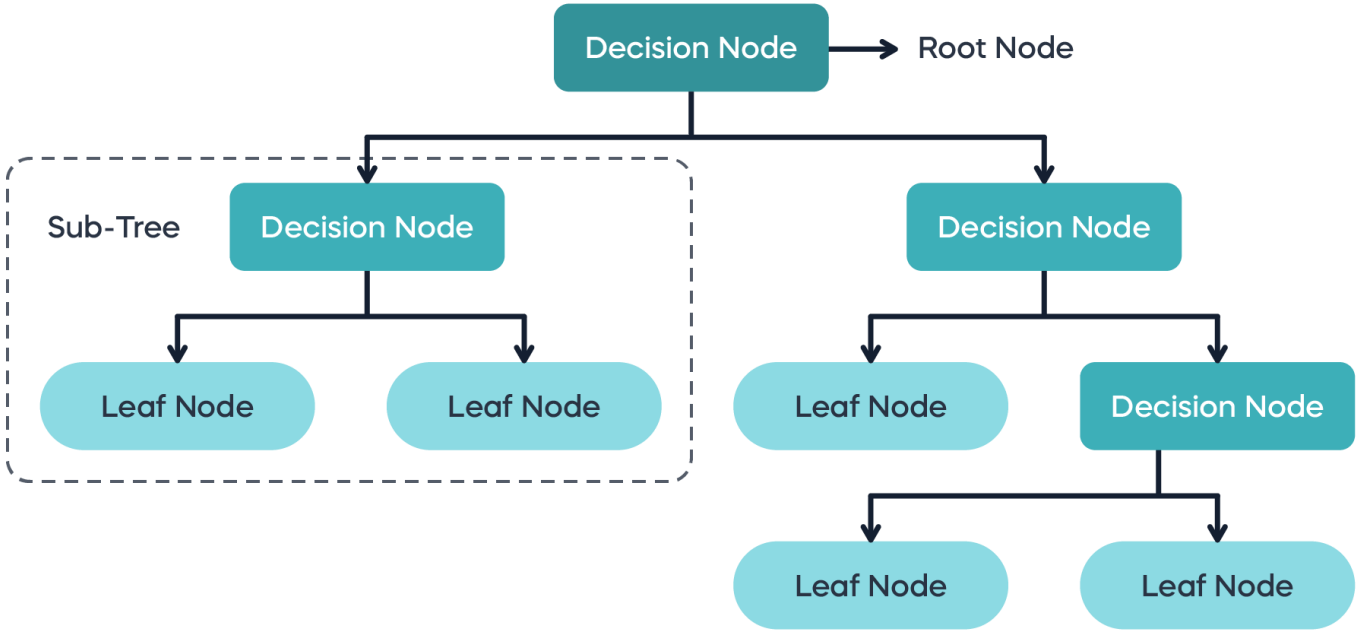
- ML Fundamentals
- Data Processing
- Tabular ML
- Lab

Why Tabular ML Matters

- Most real-world data is tables, not images or text
- Structured data dominates: finance, healthcare, logistics, marketing, HR
- Deep learning is rarely the best choice here: tree-based methods typically win
- Fast to train, easy to interpret, strong baselines

Decision Trees

- Split the data on feature thresholds, one question at a time
- Easy to visualize and explain
- Problem: a single tree overfits easily
- Solution: combine many trees

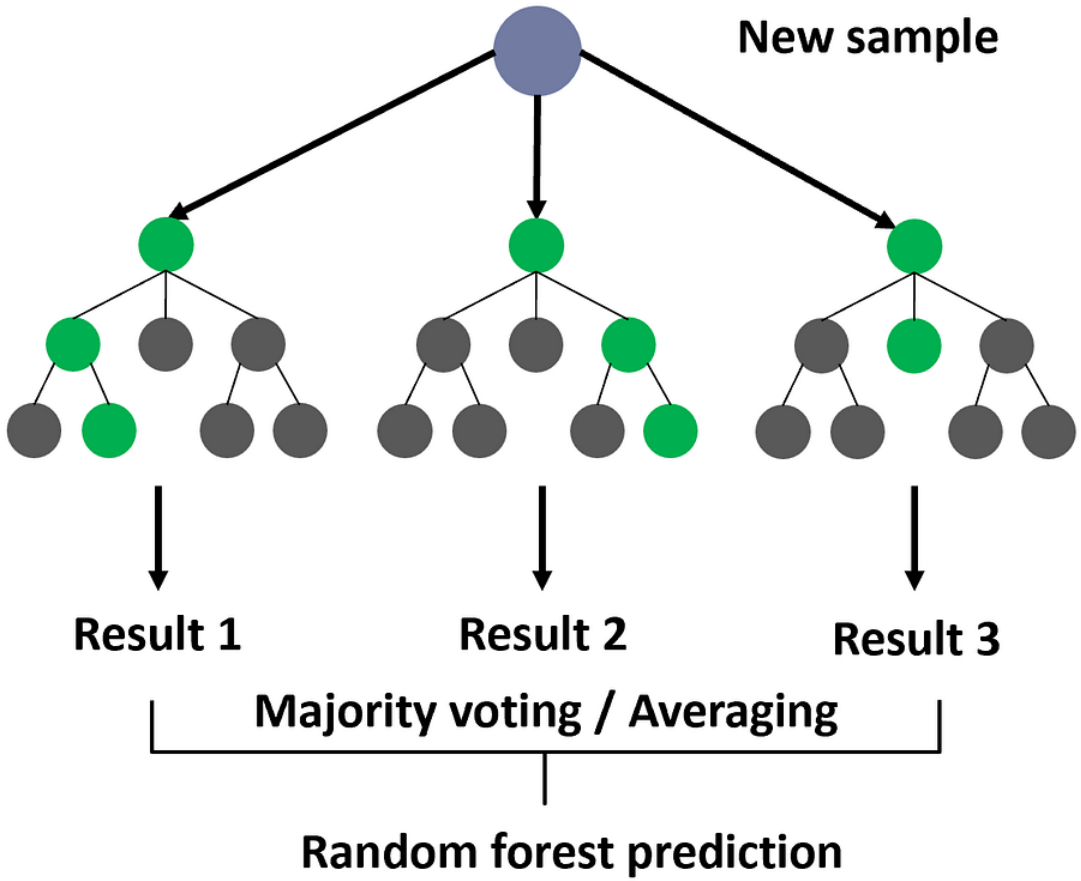


(Example)

Day	Condition	Humidity	Go to lecture?
D1	Sunny	High	No
D2	Sunny	High	No
D3	Overcast	High	Yes
D4	Rain	Normal	Yes
D5	Rain	High	No
D6	Sunny	Normal	Yes

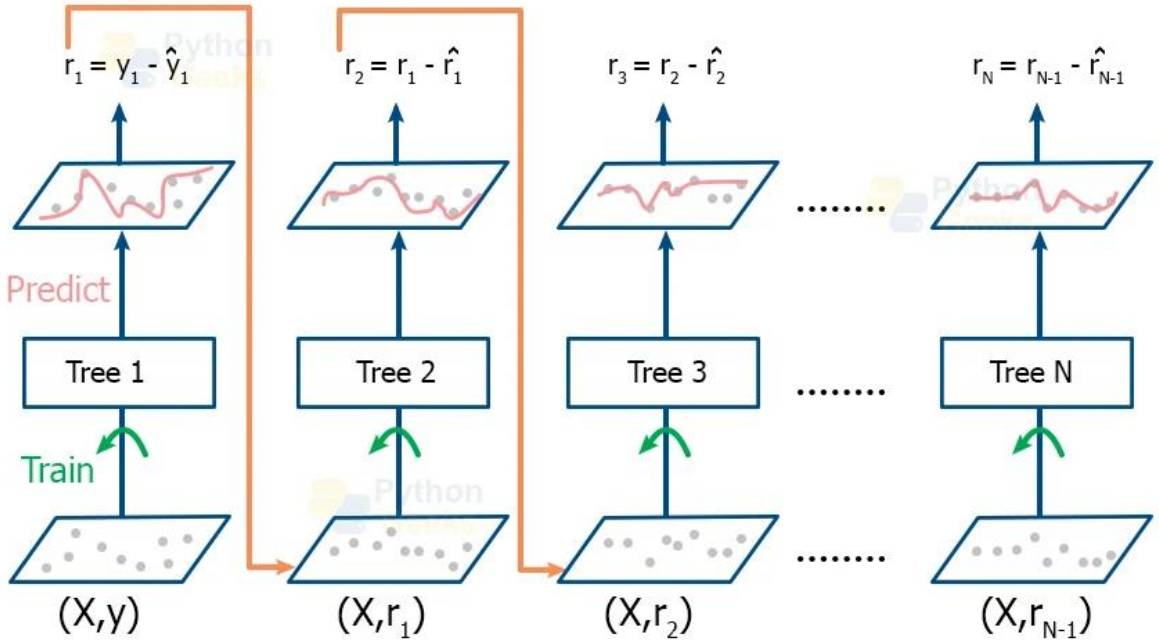
Random Forests

- Build many decision trees, each on a random subset of data and features
- Final prediction: majority vote (classification) or average (regression)
- Key idea: individual trees are weak and noisy, but the ensemble is strong
- Hyperparameters: number of trees, max depth, min samples per leaf



Gradient Boosting

- Build trees sequentially: each new tree corrects the errors of the previous ones
- Extremely powerful on tabular data
- Optimized versions: XGBoost, LightGBM, CatBoost
- More prone to overfitting than random forests: needs careful tuning



Overview

- ML Fundamentals
- Data Processing
- Tabular ML
- Lab

Lab

- End-to-end tabular ML pipeline
- Load a dataset and explore it
- Clean the data, engineer features
- Train a random forest and a gradient boosting model

ขอบคุณครับ

Thank you!

etienne@cmkl.ac.th